National University of Sciences & Technology
School of Electrical Engineering and Computer Science
Department of Computing

CS 417: Large Language Models (2+1)
Fall 2026

ASSIGNMENT 1

Building an Urdu Byte-Pair Encoding Tokenizer from Scratch

Assignment Report

Group Members

No.	Name	Registration No.
1	Muhammad Ahmad	479599
2	Muhammad Hassan	453837
3	Zakriya Mansoor	458368
4	Mahad Jawad	453833

Submitted to: Dr. Naima Iltaf

Code repository: https://github.com/muahmad232/urdu-bpe-tokenzier
Large files: https://drive.google.com/drive/folders/14C3K_xg9nIOMjC6ATKpUZY9NBh_QY7PB

Urdu BPE Tokenizer from Scratch

1. Overview

This report describes the byte-level BPE tokenizer we built for Urdu without using any tokenizer library. We collected and cleaned a multi-genre corpus (over 600 MB raw), wrote the Unicode normaliser and the tokenizer ourselves, trained a vocabulary of exactly 50,000 tokens, and then compared it with GPT-2 and `o200k_base` on held-out text. Sections 3 to 5 answer the three analysis questions of Part C, and Section 6 lists what we are submitting and how to rebuild it.

Design choice	What we did
Base vocabulary	IDs 0–255 are the raw byte values, so every string can be encoded and decoded without loss and there is never an unknown token.
Special token	`<|endoftext|>` has ID 256 and is kept out of every merge.
Merges	49,743 merges, IDs 257 to 49,999, stored in the order they were learned.
Tie-breaking	When two pairs have the same count we merge the lexicographically greater pair (id1, id2), which makes training deterministic.
Pre-tokenisation	A custom regex (`regex` module) that keeps combining marks (`\p{M}`) and ZWNJ (U+200C) inside the word, so diacritics do not split Urdu words the way GPT-2's pattern does. A leading space stays attached to the following word.
Training	Two stages. A reference version recounts every pair after each merge and was used for checking. The fast version counts each distinct word once, keeps a pair-to-words index so only affected words are updated, and caches word encodings. The fast version was used for the full run.
Normalisation	Done in `normalize.py`, outside the tokenizer, and idempotent (normalising twice gives the same text).

Table 1. *Main design decisions.*

2. Corpus and Training Setup

We combined three sources from three genres. About 52.8 MB came from our own scraping of three news sites (more than 2,200 articles); the rest comes from published corpora, as the assignment allows.

Source	Genre	Raw size (MB)
Daily Express, Daily Jang, BBC Urdu (self-scraped)	News	52.76
Makhzan	Classical literature	47.82
CC-100, Urdu portion (streamed)	Web text	502.58
Total		**603.15**

Table 2. *Corpus sources before cleaning.*

Every document went through `normalize.py` and a check on Urdu-script density, and exact duplicates were removed with SHA-256 hashes, leaving 195,086 documents. We shuffled with a fixed seed (42) and split by document: `train.txt` is 566.65 MB (191,184 documents), `val.txt` is 6.20 MB and `test.txt` is 5.26 MB. Table 3 gives the details of the training run.

Item	Value
Machine	Intel64 Family 6 Model 142 Stepping 10 (GenuineIntel), 8 cores, 7.85 GB RAM, Windows 11 AMD64, Python 3.13.7
Training data	335,145,273 characters in 191,184 documents; 627,748 distinct pre-tokens (unique word types)
Training time	425.29 s (7 min 5 s) wall-clock for 49,743 merges, about 117 merges per second
Encoded output	`train.bin` holds 81.1 M token IDs and `val.bin` holds 885.8 k (uint16)

Table 3. *Training environment and results.*

3. Question 1: What Did the Merges Learn?

Every letter in the Arabic-script block (U+0600–U+06FF) takes two bytes in UTF-8: a lead byte between 0xD8 and 0xDB and a continuation byte. Before the tokenizer can learn anything word-like, it has to glue these byte pairs back into letters. Table 4 shows when that happens for the 18 most frequent letters. A merge's token ID is simply 256 plus its merge number.

Letter	Code pt	Merge	Bytes	Letter	Code pt	Merge	Bytes	Letter	Code pt	Merge	Bytes
ا	U+0627	#1	D8 A7	ے	U+06D2	#9	DB 92	س	U+0633	#21	D8 B3
ی	U+06CC	#2	DB 8C	ن	U+0646	#11	D9 86	د	U+062F	#22	D8 AF
ک	U+06A9	#4	DA A9	ل	U+0644	#13	D9 84	ھ	U+06BE	#24	DA BE
ر	U+0631	#6	D8 B1	م	U+0645	#14	D9 85	گ	U+06AF	#26	DA AF
و	U+0648	#7	D9 88	ں	U+06BA	#15	DA BA	ب	U+0628	#32	D8 A8
ہ	U+06C1	#8	DB 81	ت	U+062A	#18	D8 AA	ف	U+0641	#41	D9 81

Table 4. *Two-byte Urdu letters rebuilt by the first 41 merges.*

Alif is rebuilt first, then choti ye, kaf and re, which roughly follows how often each letter occurs in running Urdu. Rarer letters come much later; zhe (ژ) is not merged until merge 1,220.

#	ID	Pair	Token	Valid UTF-8
1	257	[216, 167]	ا	yes
2	258	[219, 140]	ی	yes
3	259	[32, 216]	`20 D8`	no
4	260	[218, 169]	ک	yes
5	261	[32, 217]	`20 D9`	no
6	262	[216, 177]	ر	yes
7	263	[217, 136]	و	yes
8	264	[219, 129]	ہ	yes
9	265	[219, 146]	ے	yes
10	266	[32, 260]	space + ک	yes
11	267	[217, 134]	ن	yes

Table 5. *The first eleven merges.*

Merge	ID	Token	Bytes	Meaning
5,000	5256	اسٹیٹ	11	state
10,000	10256	ریسٹ	9	rest (loan word)
15,000	15256	رحمانی	13	Rahmani (name)
20,000	20256	فورٹ	9	fort
25,000	25256	پچھلا	11	previous
30,000	30256	سائنسداں	17	scientist
35,000	35256	حدیثِ	11	hadith (with izafat)
40,000	40256	(	4	punctuation
45,000	45256	Republic	9	English word

Table 6. *The token created at every 5,000th merge.*

Merges 3 and 5 are not letters: they join a space with the lead byte of the next letter (0xD8 or 0xD9). That is why they are not valid UTF-8 on their own. In total 114 of the 49,743 merged tokens are not valid UTF-8 when decoded alone (examples in Table 7). They are by-products of building letters and punctuation out of bytes, and they act as bridges that let later merges create space-prefixed words in one step. Merge 12, for instance, is alif plus the lead byte of the next letter, merge 38 adds a space in front of it, and merge 100 is 0xE2 0x80, the first two bytes of three-byte punctuation such as curly quotes and dashes.

The first whole word is کی (“of”) at merge 28 (ID 284), stored together with its leading space. Next come کے at merge 29, میں at merge 40, کر at merge 46 and ہے at merge 48, so within the first 50 merges the vocabulary already contains the most common Urdu function words. Table 6 shows the general trend after that: short words and function words early, whole nouns by merge 5,000, names and longer words between 15,000 and 30,000, and a mix of punctuation, inflected forms and English words after 40,000.

Merge	ID	Composition	Bytes
3	259	space + 0xD8	`20 D8`
5	261	space + 0xD9	`20 D9`
12	268	alif + 0xD8	`D8 A7 D8`
38	294	space + (alif + 0xD8)	`20 D8 A7 D8`
100	356	punctuation prefix	`E2 80`

Table 7. *Examples of tokens that are not valid UTF-8.*

Rank	ID	Token	Bytes	Chars
1	28048	run of ۔	64	32
2	42536	run of ۔	52	26
3	44360	run of hyphens	43	43
4	13450	run of ۔	32	16
5	47863	run of kasratan marks	32	16
6	48808	اَلْمِ◌ٍ◌ٍ◌ُ◌ْ ِوِكِيْنَ	31	16
7	49908	اَلْمُؤِ◌ْمِنِيْنَ	31	16

Table 8. *The seven longest tokens.*

The longest tokens are not words at all. Ranks 1 to 4 are runs of the Urdu full stop or of hyphens, decorative separator lines that appear in scraped pages, rank 5 is a run of kasratan marks, and ranks 6 and 7 are heavily vowelled Arabic phrases that look like quotations from religious text. Figure 1 shows the overall picture: token length starts at two bytes and then settles at an average of roughly 8 to 12 bytes.

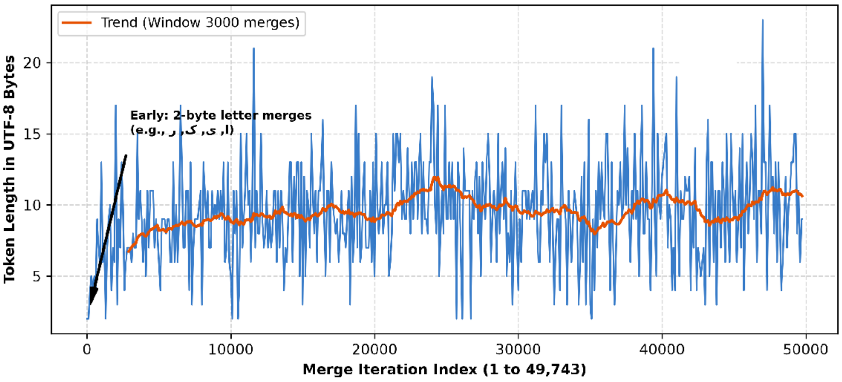

Figure 1. *Length in UTF-8 bytes of the token created at each merge (blue) with a 3,000-merge moving average (orange).*

4. Question 2: Comparison with Existing Tokenizers

We compared our tokenizer with GPT-2 (`gpt2`, 50,257 tokens) and OpenAI’s multilingual `o200k_base` (200,000 tokens), using `tiktoken` for the two reference encodings only. The held-out test split has 1,952 documents, 3,112,488 characters and 649,343 words (5.23 MB of text).

Tokenizer	Vocab	Total tokens	Tokens per word	Bytes per token	vs GPT-2
Ours (Urdu BPE)	50,000	**749,843**	**1.155**	**7.32**	**5.24× shorter**
OpenAI o200k_base	200,000	1,038,413	1.599	5.29	3.78× shorter
OpenAI GPT-2	50,257	3,928,540	6.050	1.40	baseline

Table 9. *Results on the full held-out test split.*

On the test split our tokenizer needs 1.155 tokens per word, against 6.05 for GPT-2, so sequences are about 5.2 times shorter (81% fewer tokens). It also produces 27.8% fewer tokens than `o200k_base` with a quarter of its vocabulary. This comparison is not entirely fair, though: our merges were learned on text from the same sources as the test split, while `o200k_base` is a general multilingual tokenizer. The numbers show how well the vocabulary fits Urdu text of this kind, not a general ranking.

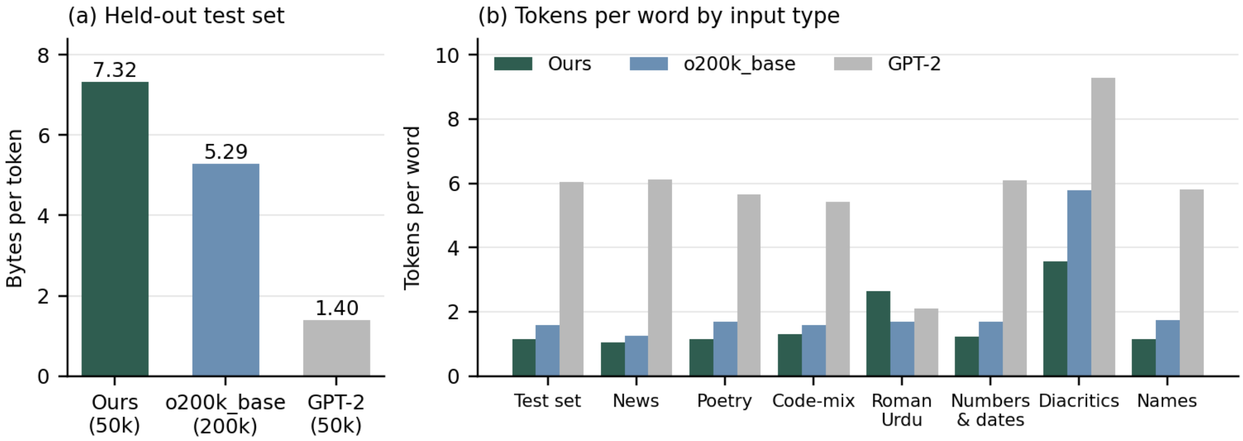

Figure 2. *(a) Bytes per token on the test split (higher is better). (b) Tokens per word for the test split and the seven hand-picked inputs (lower is better).*

For the second half of the question we chose seven short inputs, one per category listed in the assignment. Table 10 gives the token count and, in brackets, tokens per word.

Input type	Snippet	Words	Ours	o200k_base	GPT-2
News	قومی اسمبلی میں آئینی ترمیم کے حوالے سے…	27	**28 (1.04)**	34 (1.26)	165 (6.11)
Poetry	ہزاروں خواہشیں ایسی کہ ہر خواہش پہ دم نکلے…	37	**42 (1.14)**	62 (1.68)	209 (5.65)
Urdu + English	آج کل کے دور میں Artificial Intelligence ا…	26	**34 (1.31)**	41 (1.58)	141 (5.42)
Roman Urdu	*Mujhe lagta hai ke yeh naya machine learni…*	28	74 (2.64)	**47 (1.68)**	59 (2.11)
Numbers, dates	پاکستان کا قیام ۱۵ اگست ۱۹۴۷ کو عمل میں…	23	**28 (1.22)**	39 (1.70)	140 (6.09)
Diacritics	اَلْحَمْدُ لِلّٰهِ رَبِّ الْعٰلَمِيْنَ …	14	**50 (3.57)**	81 (5.79)	130 (9.29)
Personal names	ڈاکٹر علامہ محمد اقبال، مرزا اسد اللہ خان…	26	**30 (1.15)**	45 (1.73)	151 (5.81)

Table 10. *Tokens (tokens per word) on the hand-picked inputs; the best result in each row is in bold. Snippets are shortened.*

News, poetry and personal names stay between 1.04 and 1.15 tokens per word, and Urdu mixed with English costs a little more at 1.31. Fully vowelled text is the hardest case for us (3.57), although the other two tokenizers do worse on it. The one place where we lose is Roman Urdu: 2.64 tokens per word against 1.68 for `o200k_base` and 2.11 for GPT-2. That is what we expected, since the whole corpus is in Perso-Arabic script.

5. Question 3: Inputs the Tokenizer Handles Badly

Case 1: Roman Urdu with stretched letters

> *"yrrr plzzzzz mujhy btao ye assignment kb tk submit krwani h wrna scene off hojana"*

15 words. Ours: 42 tokens (2.80 per word) o200k_base: 25 GPT-2: 29. Our split: yr | r | r | pl | z | z | z | z | z | m | uj | hy …

Our corpus is entirely Perso-Arabic script, so the vocabulary has almost no merges for Latin letter combinations. Informal chat spelling with repeated letters ("yrrr", "plzzzzz") therefore falls back to single ASCII bytes, and the result is worse than both reference tokenizers.

Case 2: Heavily vowelled Arabic-style text

مُسْتَشْفَيَاتٌ (Mustashfayatun) بِالْإِضَافَةِ إِلَى تَسْخِيرِ الطَّاقَاتِ

6 words. Ours: 39 tokens (6.50 per word) o200k_base: 49 GPT-2: 77.

Our pattern keeps combining marks attached to their word, so they are not cut off during pre-tokenisation, but BPE can only merge combinations it saw often. Most modern Urdu is written without short-vowel marks, so letter-plus-mark combinations rarely reached the frequency needed. Nearly every mark ends up as its own two-byte token, which gives 6.5 tokens per word compared with about 1.1 on ordinary prose.

Case 3: Honorific signs and rare ligatures

نبی کریم ﷺ اور صحابہ کرامؓ وؑ کے ارشادات مبارکہ

12 words. Ours: 12 tokens o200k_base: 22 GPT-2: 59.

This sentence is handled well, and common words become single tokens, but it shows where the weak spot is. The honorific ligature ﷺ (U+FDFA) is a single code point that takes three bytes, and the signs ؓ (U+0613) and ؑ (U+0611) take two bytes each. Rarer ligatures such as ﷽ (U+FDFD) are not frequent enough to earn merges; in our tests they fell back to byte pieces. A corpus with more religious text, or an explicit step that keeps these symbols whole, would help.

6. Deliverables and Reproduction

The archive contains `normalize.py` (normaliser), `bpe.py` (the `BPETokenizer` class with the reference and fast trainers), `train_tokenizer.py` (training runner that also records the hardware details), `encode_corpus.py` (streaming encoder with a word cache that writes the uint16 files), `tokenizer.json` (vocabulary and all 49,743 merges), `DATA_CARD.md`, `README.md` and this report. The text corpus and the binary files are on the shared drive (links on the cover page). To rebuild everything from `train.txt`:

```
# 1. unit tests
python tests/test_normalize.py
python tests/test_bpe.py

# 2. train the 50,000-token vocabulary
python train_tokenizer.py --input corpus/train.txt --vocab-size 50000 --output tokenizer.json

# 3. encode the corpus to flat uint16 arrays
python encode_corpus.py --tokenizer tokenizer.json --train-in corpus/train.txt \
    --val-in corpus/val.txt --train-out data/train.bin --val-out data/val.bin

# 4. regenerate the Part C analysis
python scripts/analyze_tokenizer.py
```